

Ecosistema de Automatización IA

Autónomo para Negocios

Entrega final (intento 2) · Agustina Cisneros

Caso de uso: pipeline de generación y publicación de contenido con IA (RAG), con aprobación humana en tiempo real.

Orquestador	n8n: un solo workflow con 4 subworkflows internos (Execute Workflow → mismo workflow)
Base de datos	Airtable: Contenidos ⇄ Log de Errores (vinculadas) y Manual de Marca (RAG)
IA	Google Gemini con RAG y prompt dinámico (se puede cambiar por Claude u OpenAI)
Salida	Slack: aprobación HITL con botones, publicación y respuestas en hilo (thread_ts)

Enlaces

Workflow n8n (.json)	https://github.com/aguscisneros1/entrega-final/blob/claude/ia-automation-unified-workflow-iuscap/workflow/ecosistema-contenido-ia.n8n.json
Repositorio	https://github.com/aguscisneros1/entrega-final/tree/claude/ia-automation-unified-workflow-iuscap
Dashboard de control (Shared View)	https://airtable.com/appCtyPCA4QGzyXR5/shr9RbSHu2YsiSJFt
Base de datos (lectura)	https://airtable.com/appCtyPCA4QGzyXR5/shrBIC22XsubHnrfF
Video demo (3 min)	https://drive.google.com/file/d/1jWk3e47hR2khTUuzZeiAwceMCg9LZW8d/view?usp=sharing

Correcciones respecto del intento 1

- **Un solo workflow con subworkflows:** los 3 escenarios de Make se reemplazaron por un único workflow de n8n que invoca SUB 1 Generación RAG, SUB 2 Publicación, SUB 3 Rechazo y SUB 4 Registro de errores.
- **HITL con pausa real:** el nodo Slack *Send and Wait for Response* detiene la ejecución (estado *Waiting*) hasta que una persona aprieta Aprobar o Rechazar. Recién ahí continúa.
- **Repositorio completo:** el workflow, las instrucciones de ejecución, la documentación de los 5 criterios, el plan de pruebas y las evidencias.

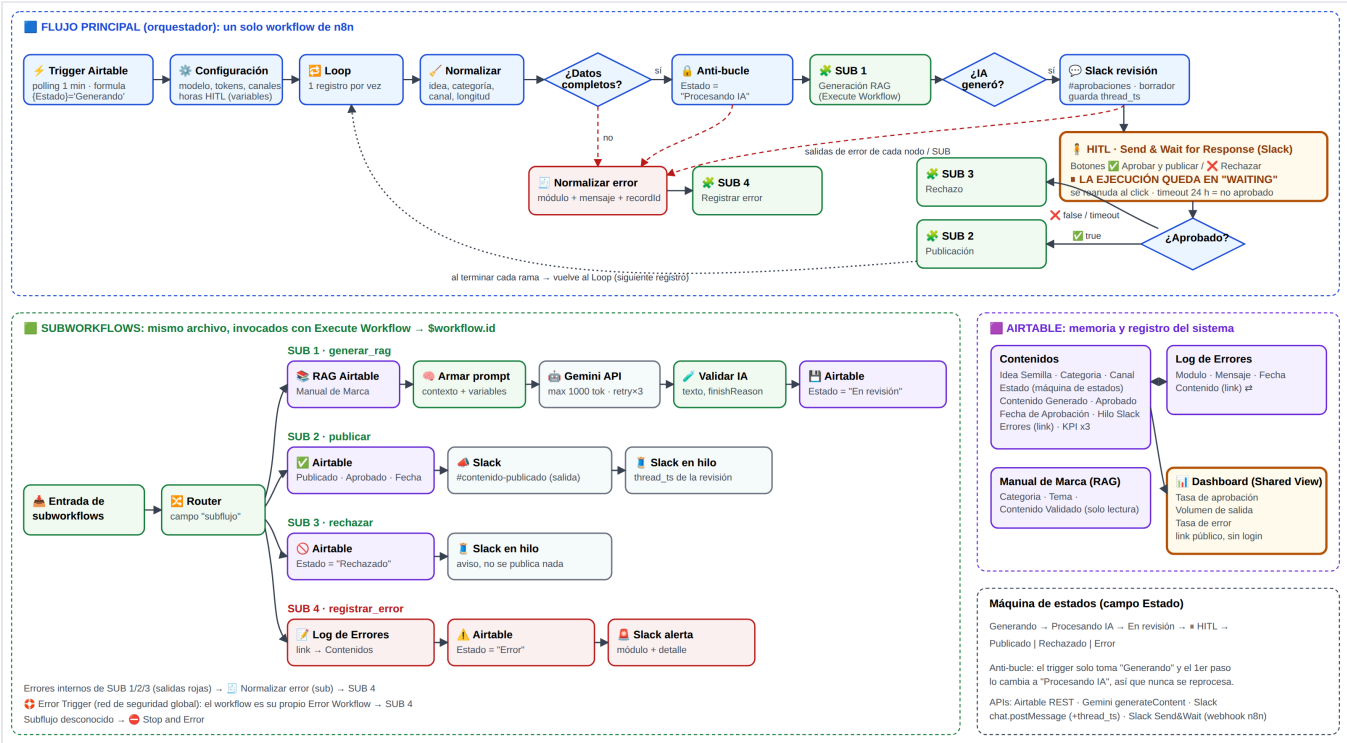
1. Mapa de arquitectura

Proyecto: Pipeline de generación y publicación de contenido con IA **Orquestador:** n8n, en un solo **workflow** con 4 subworkflows internos y un punto HITL que **pausa la ejecución** **Archivo técnico:** workflow/ecosistema-contenido-ia.n8n.json

1.1 Qué cambió respecto del primer intento

Devolución del profesor	Cómo se resolvió
"Se espera que la integración suceda en un solo flujo de trabajo con subworkflows y no tres flujos separados"	Los 3 escenarios de Make se unificaron en un único workflow de n8n . El flujo principal invoca 4 subworkflows con el nodo <i>Execute Workflow</i> , que llama al mismo workflow (\$workflow.id). El nodo <i>Entrada de subworkflows</i> recibe la llamada y un <i>Router</i> elige el subflujo. Se importa como un solo archivo .json.
"Revisá la integración de HITL: se debe detener el flujo y esperar la intervención humana en tiempo real"	El nodo HITL · Esperar aprobación humana (Slack → <i>Send and Wait for Response</i>) pausa la ejecución . El workflow no avanza hasta que una persona aprieta Aprobar y publicar o Rechazar en Slack. Recién en ese momento continúa por la rama correspondiente. Antes, el "HITL" era un checkbox en Airtable que disparaba otro escenario. Ahora la espera ocurre dentro de la misma ejecución.
"Asegurar que el repositorio contenga los archivos, configuración y documentación. Agregar instrucciones de ejecución y evidencias verificables"	El repositorio incluye el workflow, un README con los pasos de ejecución, la documentación de los 5 criterios en docs/, un plan de pruebas y una lista de evidencias a capturar.

1.2 Diagrama



El mismo diagrama, en versión visual para imprimir, está en entrega/Diagrama-Arquitectura.pdf.

1.3 Componentes por categoría obligatoria

Categoría	Tecnología	Nodos
Orquestador	n8n: 1 workflow, 4 subworkflows	< Trigger,  Loop,  Router de subworkflows,  SUB 1..4 (Execute Workflow)
Base de datos (memoria)	Airtable: base "Pipeline de Contenido" y base "Manual de Marca"	Contenidos (estado), Log de Errores (vinculada a Contenidos), Manual de Marca (RAG)
Procesamiento IA	Google Gemini gemini-3.6-flash vía API REST, con RAG y prompt estructurado. Se puede cambiar por Claude u OpenAI: ver criterio 3	 Unir contexto + armar prompt,  IA · Gemini genera borrador,  Validar respuesta IA
Canal de salida	Slack: aprobaciones con botones, publicación y respuestas en hilo (thread_ts)	 Slack · Enviar pieza a revisión,  HITL,  Slack · Publicar,  Respuestas en hilo,  Alerta de error

1.4 Recorrido de una pieza (camino feliz)

1. En Airtable se carga una fila con *Idea Semilla*, *Categoría*, *Canal* y **Estado = Generando**.
2. El **trigger de Airtable** consulta la tabla cada minuto y solo trae registros que cumplen {Estado} = 'Generando'. Así se evitan operaciones innecesarias.
3.  Configuración define las variables del sistema: modelo, max tokens, temperatura, canales y horas de espera del HITL. Ningún dato de negocio queda escrito dentro de los nodos.
4. El **loop** procesa **un registro por vez**. Cada registro tiene su propia pausa HITL.
5. Se valida la entrada. Enseguida el registro pasa a **Procesando IA**, que funciona como bloqueo anti-bucle: el trigger ya no vuelve a tomarlo.
6. **SUB 1** busca el contexto del Manual de Marca (RAG), arma el prompt dinámico, llama a Gemini (con 3 reintentos ante un 503), valida la respuesta y guarda el borrador en **En revisión**.
7. Slack recibe la pieza completa en #aprobaciones-contenido. Su thread_ts se guarda para responder en el mismo hilo.
8. **HITL**: el workflow **se pausa** con los botones Aprobar/Rechazar. Si nadie responde en 24 h (configurable), se toma como rechazo.
9. Si se aprueba, **SUB 2** marca **Publicado**, completa la *Fecha de Aprobación* y el *Hilo Slack*, publica en #contenido-publicado y responde en el hilo original.
10. Si se rechaza, **SUB 3** marca **Rechazado**, responde en el hilo y no contacta a nadie.
11. Si algo falla, **SUB 4** crea un registro en **Log de Errores** vinculado al contenido, marca **Error** y manda una alerta a Slack. Después el loop sigue con el próximo registro.

2. Manual operativo de datos

2.1 Esquema de tablas en Airtable

El sistema usa la base **"Pipeline de Contenido"** (appCtyPCA4QGzyXR5) como memoria y registro. El contexto RAG sale de la base **"Manual de Marca"** (app8saP7sKupgcjAK), que el flujo solo lee.

Tabla Contenidos (tb1PQPkpH8S5B10wz): núcleo y máquina de estados

Campo	Tipo	Quién lo escribe	Uso
Idea Semilla	Long text	Humano	Tema que desarrolla la IA (obligatorio, mínimo 10 caracteres)
Categoría	Single select	Humano	Filtra los fragmentos del Manual de Marca (RAG)
Canal	Single select	Humano	Destino editorial (ej. LinkedIn). Se incluye en el prompt
Estado	Single select	Humano / n8n	Generando → Procesando IA → En revisión → Publicado / Rechazado / Error
Contenido Generado	Long text	n8n (SUB 1)	Borrador de la IA
Aprobado	Checkbox	n8n (SUB 2/3)	Resultado de la decisión humana tomada en Slack
Fecha de Aprobación	Date	n8n (SUB 2)	Fecha en que se aprobó
Hilo Slack	Single line text	n8n (SUB 2/3)	thread_ts del mensaje de revisión, para trazabilidad del hilo
Notificado	Checkbox	(legado Make)	Ya no se usa: la notificación y la espera ocurren dentro de la misma ejecución
Ultima Modificacion	Last modified time	Airtable	Campo que usa el trigger de n8n
Errores	Link → Log de Errores	Airtable (campo inverso)	Relación 1:N con los errores del registro
KPI Aprobación / KPI Salida / KPI Error	Formula	Airtable	Alimentan el Dashboard (ver criterio 5)

Los estados equivalen a los que pide la consigna: *Pendiente* = Generando, *Procesado por IA* = En revisión, *Aprobado por Humano* = Publicado.

Tabla Log de Errores (tb1gYFeGeFImx8eZe): resiliencia y auditoría

Campo	Tipo	Uso
Modulo	Single line text	Nodo de n8n que falló (ej. 🤖 IA · Gemini genera borrador)
Mensaje	Long text	Mensaje de error capturado
Fecha	Created time	Automático
Contenido	Link → Contenidos	Relación entre tablas: el error queda vinculado al registro afectado, así ningún dato queda aislado

Tabla Manual de Marca (tblJNVng0B0StcL4k, vista verificada viwLVKJmk27LL5Wy8): fuente RAG, solo lectura

Campo	Tipo	Uso
Categoría	Single select	Se filtra con {Categoría} = '<categoría del contenido>'
Tema	Single line text	Título del fragmento
Contenido Validado	Long text	Texto que se inyecta como contexto en el prompt

Esta tabla vive en otra base y Airtable no permite links entre bases. Por eso la relación con Contenidos es lógica: la hace el flujo, por **Categoría**. La sincronización entre ambas bases también la resuelve el flujo, como pide la consigna.

Cambios a aplicar en Airtable (una sola vez)

1. En Contenidos, agregar el campo **Hilo Slack** (Single line text).
2. En Log de Errores, agregar el campo **Contenido** (Link to another record → Contenidos). Airtable crea solo el campo inverso **Errores** en Contenidos.
3. Las opciones Procesando IA y Error del campo **Estado** se crean solas: los nodos de Airtable escriben con typecast: true. También se pueden crear a mano.
4. Crear los 3 campos fórmula del dashboard (criterio 5).

2.2 Esquemas JSON de transferencia entre integraciones

Todos los ejemplos son reales en estructura. Los IDs se acortaron.

a) Airtable Trigger → Configuración (entrada del flujo)

```
{
  "id": "recECFm6VtGe07RrI",
  "createdTime": "2026-09-29T13:02:11.000Z",
  "fields": {
    "Idea Semilla": "Tips de organización para freelancers",
    "Categoría": "Servicios",
    "Canal": "LinkedIn",
    "Estado": "Generando",
    "Ultima Modificacion": "2026-09-29T13:02:40.000Z"
  }
}
```

b) Normalizar registro: objeto interno del flujo

```
{
  "recordId": "recECFm6VtGe07RrI",
  "idea": "Tips de organización para freelancers",
  "categoria": "Servicios",
  "canal": "LinkedIn",
  "longitudIdea": 37,
  "cfg": {
    "modeloIA": "gemini-3.6-flash", "maxTokens": 1000, "temperatura": 0.7,
    "maxPalabras": 200, "minCaracteresIdea": 10,
    "canalAprobaciones": "C0C12LQCXJR", "canalPublicacion": "C0C1128NG67",
  }
}
```

```

    "canalAlertas": "C0C12LQCXJR", "horasEsperaHITL": 24
  }
}

```

longitudIdea y minCaracteresIdea son **números** y se comparan como números (number ≥ number), con validación de tipos estricta en el IF.

c) Contrato de llamada a un subworkflow (Execute Workflow → Entrada de subworkflows)

```

{ "subflujo": "generar_rag", "recordId": "rec...", "idea": "...", "categoria": "Servicios",
  "canal": "LinkedIn", "cfg": { ... } }

```

Valores posibles de subflujo: generar_rag | publicar | rechazar | registrar_error. Si llega otro valor, el Router lo manda a  Subflujo desconocido, que corta con un error.

d) Airtable Search (Manual de Marca) → Unir contexto + armar prompt

```

{ "id": "recA1...", "Categoria": "Servicios", "Tema": "Consultoría", "Contenido Validado":
  "Ofrecemos consultoría personalizada de organización financiera para freelancers...",
  { "id": "recA2...", "Categoria": "Servicios", "Tema": "Tono de voz", "Contenido Validado":
    "Hablamos de vos, cercano y profesional..." }
}

```

Puede devolver N fragmentos. El nodo Code los concatena en un solo bloque de contexto. Si devuelve 0, se toma como error: no se gastan tokens sin contexto.

e) Request a la IA (HTTP → generateContent de Gemini)

```

{
  "system_instruction": { "parts": { "text": "Sos el redactor de contenido de la marca. Tu
  única fuente de verdad es el CONTEXTO VALIDADO... de no más de 200 palabras." } },
  "contents": { "role": "user", "parts": [{ "text": "Idea semilla: Tips de organización
  para freelancers\nCategoría: Servicios\nCanal destino: LinkedIn\n\nCONTEXTO VALIDADO DE LA
  MARCA:\nTema: Consultoría\nOfrecemos..." } ]},
  "generationConfig": { "maxOutputTokens": 1000, "temperature": 0.7 }
}

```

Todo el prompt se arma con variables (idea, categoria, canal, contexto RAG, cfg.maxPalabras). No hay texto de negocio escrito a mano.

f) Respuesta de la IA → Validar respuesta IA

```

{
  "candidates": { "content": { "parts": [{ "text": "¿Sos freelancer y sentís que tu tiempo
  no rinde?..." } ]}, "finishReason": "STOP" }},
  "usageMetadata": { "promptTokenCount": 812, "candidatesTokenCount": 356 }
}

```

Se mapea candidates0.content.parts*.text a contenido. Además se calculan palabras, finishReason y los tokens de entrada y salida, que sirven para controlar costos.

g) Salida de SUB 1 (vuelve al flujo principal)

```
{ "ok": true, "recordId": "rec...", "idea": "...", "categoria": "Servicios", "canal": "LinkedIn", "contenido": "¿Sos freelancer...", "palabras": 187, "tokensEntrada": 812, "tokensSalida": 356 }
```

h) Slack: mensaje de revisión (chat.postMessage)

```
{ "channel": "C0C12LQCXJR", "text": "📄 *Nueva pieza para revisar* (registro `rec...`)\n*Idea semilla:* ...\n*Borrador generado por IA:*>>> ..." }
```

Respuesta: { "ok": true, "channel": "C0C12LQCXJR", "message_timestamp": "1759150000.123456" }. Ese valor es el **Thread ID** (thread_ts) que se reutiliza después.

i) HITL: Slack *Send and Wait for Response* (salida cuando el humano responde)

```
{ "data": { "approved": true } }
```

approved: false (❌ Rechazar), o que se cumpla el plazo sin respuesta, lleva a SUB 3.

j) Payload de SUB 2 / SUB 3 y respuesta en hilo

```
{ "subflujo": "publicar", "recordId": "rec...", "idea": "...", "canal": "LinkedIn", "contenido": "...", "threadTs": "1759150000.123456", "cfg": { ... } }
```

```
{ "channel": "C0C12LQCXJR", "thread_ts": "1759150000.123456", "text": "✅ Aprobado por revisión humana y publicado en #contenido-publicado..." }
```

k) Registro final en Airtable (tras aprobación)

```
{ "id": "rec...", "fields": { "Estado": "Publicado", "Aprobado": true, "Fecha de Aprobación": "2026-09-30", "Hilo Slack": "1759150000.123456", "Contenido Generado": "..." } }
```

l) SUB 4: registro de error

```
{ "subflujo": "registrar_error", "recordId": "rec...", "modulo": "🤖 IA · Gemini genera borrador", "mensaje": "503 - The model is overloaded", "cfg": { ... } }
```

→ Airtable Log de Errores: { "Modulo": "🤖 IA · Gemini genera borrador", "Mensaje": "503 - ...", "Contenido": "rec..." }

3. Optimización de costos: matriz de decisión de modelos

3.1 Matriz: qué modelo para cada tarea

#	Tarea del ecosistema	Volumen / latencia	Modelo elegido	Por qué	Alternativa descartada y motivo
1	Redactar el post con RAG: lectura densa del Manual de Marca y redacción con tono de marca	Bajo (pocas piezas por día); interactivo, porque hay un humano esperando para revisar	Implementado: Gemini gemini-3.6-flash (free tier). Producción: Claude Sonnet con prompt caching	En producción la prioridad es la calidad del texto y la consistencia de tono sobre contexto largo. En esta entrega se usa el free tier por presupuesto	GPT-4o-mini: más barato, pero rinde menos en redacción larga con tono de marca
2	Validar entrada (campos vacíos, largo mínimo)	Cada ejecución	Sin LLM: IF de n8n	Es una regla determinística y cuesta \$0. Usar IA acá sería desperdicio	Pedirle al LLM que "detecte datos faltantes": caro y no determinístico
3	Validar la salida de la IA (vacía, bloqueada, truncada)	Cada ejecución	Sin LLM: Code node (finishReason, conteo de palabras)	Costo \$0; mide tokens para auditar el gasto	Un segundo LLM como "juez": duplica el costo por pieza
4	Clasificar la categoría de una idea cuando llega vacía (mejora futura)	Alto volumen; texto corto	GPT-4o-mini o Claude Haiku	Tarea de bajo razonamiento: el modelo chico alcanza a un costo entre 10 y 20 veces menor	Sonnet: sobredimensionado para una clasificación
5	Regenerar el catálogo completo (ej. 1.000 piezas por cambio de manual)	Masivo; no urgente	Batch API (Anthropic u OpenAI)	50 % de descuento a cambio de respuesta diferida (hasta 24 h); no hay humano esperando	API síncrona: paga el doble sin ningún beneficio
6	Prototipado y pruebas de estrés	5 a 20 corridas	Gemini free tier	\$0; el límite de 15 req/min alcanza	—

3.2 Palancas de costo aplicadas en el workflow

Palanca	Dónde	Efecto
Trigger filtrado {Estado}='Generando '	< Trigger	El flujo solo corre cuando hay trabajo real. Sin polling de registros ya procesados
Anti-bucle Procesando IA	🔒 Anti-bucle	Un registro nunca se procesa dos veces, así que no se pagan tokens por duplicado
Validación antes de la IA	? ¿Datos completos?, ? ¿Hay contexto RAG?	Si faltan datos o no hay contexto, no se llama al modelo . En la entrega anterior una Idea Semilla vacía generaba un post inútil y gastaba tokens

Palanca	Dónde	Efecto
maxOutputTokens = 1000 y límite de maxPalabras	⚙ Configuración → prompt	Tope al costo de salida por pieza
Minimización del prompt	🧠 Unir contexto + armar prompt	Solo viajan idea, categoría, canal y el contexto filtrado por categoría, no toda la tabla
Reintentos acotados (3 × 5 s)	🤖 IA · Gemini	Resuelve el 503 que se vio en las pruebas sin reintentos infinitos
Medición de tokensEntrada / tokensSalida	🔍 Validar respuesta IA	Permite auditar el gasto real por pieza

3.3 Precios de referencia (USD por 1M de tokens)

Modelo	Input	Output	Input Batch (~50 %)	Output Batch (~50 %)
Claude Sonnet	3,00	15,00	1,50	7,50
GPT-4o-mini	0,15	0,60	0,075	0,30
Gemini Flash (pago)	1,50	7,00	0,75	3,75
Gemini free tier (usado)	0	0	—	—

Son precios de lista tomados como referencia al momento de armar la matriz. Hay que verificarlos antes de pasar a producción.

3.4 Ahorro estimado

Volumen de ejemplo: **1.000 piezas por mes**, cada una con ~800 tokens de entrada (≈500 de contexto estático del manual y ≈300 dinámicos) y ~400 de salida.

Escenario	Cálculo	Costo mensual	Ahorro vs. A
A — Sonnet estándar	$0,8 \times 3 + 0,4 \times 15$	USD 8,40	—
B — Sonnet + prompt caching (contexto estático cacheado; lectura al 10 %)	$0,3 \times 3 + 0,5 \times 0,30 + 0,4 \times 15$	USD 7,05	-16 %
C — Sonnet + caching + Batch (solo regeneración masiva)	$0,3 \times 1,5 + 0,5 \times 0,15 + 0,4 \times 7,5$	USD 3,53	-58 %
D — Clasificación (tarea 4) con GPT-4o-mini en vez de Sonnet (1.000 × 100 tokens de entrada y 10 de salida)	$0,1 \times 0,15 + 0,01 \times 0,60$ vs. $0,1 \times 3 + 0,01 \times 15$	USD 0,02 vs. 0,45	-95 % en esa tarea
E — Validaciones sin LLM (tareas 2 y 3)	Evita ~1 llamada extra por pieza	≈ USD 8,40 evitados si se usara Sonnet como juez	-100 % en esa tarea
Implementación actual — Gemini free tier	—	USD 0	-100 %

Criterio de migración: si el volumen supera el free tier (15 req/min), el primer paso es activar la facturación de Gemini Flash. Solo cambia la credencial. Si la prioridad pasa a ser la calidad de redacción, se migra la tarea 1 a Claude Sonnet con caching. Para esa migración solo se modifican 2 nodos: la URL y el body del 🤖 IA (endpoint `https://api.anthropic.com/v1/messages`, `max_tokens = cfg.maxTokens`) y el parseo en 🔍 Validar respuesta IA (`content[0].text`). El resto del workflow no cambia.

Condición para que funcione el caching: el bloque estático (system prompt y contexto del manual) tiene que ir primero y con contenido y orden idénticos en cada llamada. Cualquier variación invalida la caché.

4. Seguridad y resiliencia

4.1 Minimización de datos

Se envía a la IA	Motivo
Idea Semilla	Tema a desarrollar
Categoría	Define qué contexto RAG se usa
Canal	Adapta el formato del post
Contexto validado del Manual de Marca (solo la categoría pedida)	Única fuente de verdad permitida

Nunca viajan al modelo: Record ID, fechas, Estado, Aprobado, Hilo Slack, IDs de canales ni el resto de las filas de Airtable. El nodo  Unir contexto + armar prompt arma el request campo por campo, así que no hay forma de que se cuele el registro completo.

- **Credenciales:** las API keys (Airtable, Slack, Gemini) viven en el gestor de credenciales cifrado de n8n. El JSON del workflow no contiene ninguna key: se puede compartir sin riesgo.
- **Anti-alucinación:** el system prompt limita al modelo al contexto validado ("No uses conocimiento externo ni inventes datos").
- **Mínimo privilegio:** el token de Airtable tiene alcance solo sobre las 2 bases. El Manual de Marca solo se lee. El bot de Slack solo escribe en los canales configurados.
- **Video y capturas:** las credenciales se ocultan (se muestran solo los nombres de credencial).

4.2 Rutas de error (Error Handlers)

En n8n, lo equivalente a las directivas *Resume/Break* de Make es la opción **On Error** de cada nodo: - **Continue (using error output):** equivale a *Resume* con ruta alternativa. El error sale por una segunda salida roja y se registra. - **Continue:** equivale a *Resume/Ignore*. Se usa solo en nodos de notificación no críticos, para que un fallo al avisar no rompa el registro. - **Retry on fail:** reintento automático antes de declarar el error.

#	Nodo protegido	Falla posible	Manejo	Resultado
1	 ¿Datos completos?	Idea Semilla vacía o menor a 10 caracteres, Categoría vacía	Rama <i>false</i> → SUB 4	Log con el detalle de qué falta, Estado = Error , alerta en Slack. No se llama a la IA
2	 Anti-bucle (Airtable update)	Token vencido, registro borrado	Error output → SUB 4	Log y alerta
3	 RAG · Buscar en Manual de Marca	API de Airtable caída	Error output → SUB 4	Log y Estado = Error
4	 ¿Hay contexto RAG?	0 fragmentos para la categoría	Rama <i>false</i> → SUB 4	No se generan posts sin contexto
5	 IA · Gemini	503 por alta demanda, 429 por rate limit, timeout	Retry 3 × 5 s ; si sigue fallando, error output → SUB 4	El 503 real que apareció en las pruebas se resuelve con el reintento. Si persiste, queda registrado

#	Nodo protegido	Falla posible	Manejo	Resultado
6	? ¿Respuesta IA válida?	Respuesta vacía o bloqueada por seguridad (finishReason ≠ STOP)	Rama <i>false</i> → SUB 4	Log con el finishReason
7	📁 Guardar borrador	Error de Airtable	Error output → SUB 4	—
8	💬 Slack · Enviar pieza a revisión / 🧑 HITL	Canal inválido, token revocado	Error output → SUB 4	La pieza no queda "colgada" en revisión
9	✅ Publicado / 🚫 Publicar / ❌ Rechazado	Error de Airtable o Slack	Error output → SUB 4	—
10	Llamadas 🧩 SUB 1..3	Fallo inesperado del subworkflow	Error output → SUB 4	—
11	Cualquier nodo (error no previsto)	—	🚨 Error Trigger · red de seguridad (opcional: se activa configurando el workflow como su propio <i>Error Workflow</i> , si la versión de n8n lo permite)	Log y alerta. Ningún fallo queda silencioso

SUB 4 · Registrar error hace esto: crea el registro en Log de Errores **vinculado** al contenido, pone el contenido en Estado = Error y avisa en Slack. Los pasos no críticos usan *Continue*, así que el registrador en sí no puede fallar en cadena. Después de registrar, el loop sigue con el próximo registro.

Filtros anti-bucle y tipos de datos (check de seguridad)

- Bucles infinitos:** (a) el trigger solo trae Estado = 'Generando'; (b) el primer paso cambia el Estado a Procesando IA, así que el mismo registro no vuelve a entrar aunque su *Ultima Modificacion* cambie; (c) el loop procesa como máximo los registros del lote y termina; (d) un subflujo desconocido corta con *Stop and Error*.
- Tipos correctos:** los IF usan *type validation: strict*. longitudIdea (number) ≥ cfg.minCaracteresIdea (number); palabras (number) > 0; ok y approved (boolean) se evalúan como boolean, no como texto.
- Prompt dinámico:** el prompt se arma solo con variables del registro, del contexto RAG y de cfg.

4.3 Human-in-the-loop: pausa real

Acción crítica protegida: publicar el contenido en el canal de salida, es decir, contactar al público o cliente final.

- SUB 1 deja el borrador en **En revisión** y el flujo principal publica la pieza completa en #aprobaciones-contenido.
- El nodo 🧑 HITL · **Esperar aprobación humana** (Slack → *Send and Wait for Response*, aprobación doble) manda los botones ✅ **Aprobar y publicar** / ❌ **Rechazar**.
- La ejecución queda en estado *Waiting***. n8n guarda el estado y no ejecuta ningún nodo posterior. En la lista de ejecuciones figura como "Waiting". No hay polling ni un segundo escenario.
- Cuando la persona aprieta un botón, n8n **reanuda la misma ejecución** en tiempo real:

5. `approved = true` → **SUB 2:** Estado **Publicado**, publicación en #contenido-publicado y confirmación en el hilo.
6. `approved = false` → **SUB 3:** Estado **Rechazado** y aviso en el hilo. **No se contacta a nadie.**
7. **Timeout de seguridad:** si en `cfg.horasEsperaHITL` (24 h) nadie responde, la ejecución se reanuda como **no aprobada**. La opción segura por defecto es no publicar.

Así se evita el "**efecto metralleta**": ninguna pieza sale sin una decisión humana explícita. La trazabilidad queda en Airtable (Estado, Aprobado, Fecha de Aprobación, Hilo Slack) y en el hilo de Slack.

5. Dashboard de control

5.1 Enlace público (Shared View)

Dashboard de control (Shared View de Airtable, vista "KPIs del sistema"):

<https://airtable.com/appCtyPCA4QGzyXR5/shr9RbSHu2YsiSJFt>

⚠ Antes de entregar: abrir el link en una **ventana de incógnito**. Si pide iniciar sesión o solicitar acceso, todavía no es una Shared View.

5.2 KPIs que muestra

KPI	Definición	Cómo se calcula en Airtable
Tasa de aprobación	Publicados / (Publicados + Rechazados)	Campo fórmula KPI Aprobación y resumen Average en la barra inferior
Volumen de salida	Cantidad de piezas publicadas	Campo fórmula KPI Salida y resumen Sum
Tasa de error	Registros con Estado = Error / total procesado	Campo fórmula KPI Error y resumen Average
Pendientes de HITL	Registros en "En revisión"	Agrupación por Estado (conteo por grupo)
Errores por módulo	Conteo en Log de Errores agrupado por Modulo	Vista agrupada de Log de Errores

5.3 Cómo publicarlo (unidad "Cuadros de mando automáticos", Módulo 7)

1. En la tabla **Contenidos**, crear 3 campos de tipo *Formula*:
2. KPI Aprobación → `IF({Estado}='Publicado', 1, IF({Estado}='Rechazado', 0, BLANK()))`. Formato: porcentaje.
3. KPI Salida → `IF({Estado}='Publicado', 1, 0)`
4. KPI Error → `IF({Estado}='Error', 1, IF(OR({Estado}='Publicado', {Estado}='Rechazado'), 0, BLANK()))`. Formato: porcentaje.
5. Crear una vista **Grid** llamada KPIs del sistema:
6. Agrupar por **Estado**.
7. Mostrar solo: Idea Semilla, Categoria, Estado, Fecha de Aprobación, KPI Aprobación, KPI Salida, KPI Error y Errores (link).
8. En la **barra de resumen** (footer): KPI Aprobación → **Average** (tasa de aprobación), KPI Salida → **Sum** (volumen de salida), KPI Error → **Average** (tasa de error).
9. **Share view** → *Create a shareable link to the view* → copiar el link <https://airtable.com/app.../shr...>
10. Opcional: hacer lo mismo en **Log de Errores**, con una vista agrupada por *Modulo*.
11. Abrir el link en incógnito y sacar una captura de los 3 KPIs para las evidencias.

5.4 Por qué es un panel de control y no el link a la base

- Muestra **métricas agregadas** (tasas y volumen), no filas sueltas.
- Es de **solo lectura** y no expone campos técnicos ni credenciales.
- Se actualiza solo: cada ejecución de n8n cambia el Estado, y las fórmulas y resúmenes se recalculan al instante.

6. Test de estrés y evidencias

6.1 Plan de pruebas (mínimo 5 ejecuciones)

#	Caso	Datos de entrada en Airtable	Acción humana	Resultado esperado	Evidencia a capturar
1	Camino feliz: aprobación	Idea "Tips de organización para freelancers", Categoría "Servicios", Estado "Generando"	✓ Aprobar en Slack	Estado Publicado , Aprobado ✓, Fecha y Hilo Slack completos; post en #contenido-publicado; confirmación en el hilo	Ejecución en n8n (verde), mensaje de Slack con botones, post publicado, fila de Airtable
2	Camino feliz: rechazo	Otra idea válida	✗ Rechazar	Estado Rechazado ; aviso en el hilo; nada en #contenido-publicado	Ejecución con la rama "false" del IF
3	Pausa HITL visible	Idea válida	Esperar 2 minutos antes de responder	La ejecución figura como Waiting en <i>Executions</i> ; después de apretar el botón pasa a <i>Success</i>	Captura de la ejecución "Waiting" y de la misma ejecución terminada
4	Camino infeliz: dato faltante	Idea Semilla vacía	—	No se llama a la IA; Log de Errores "Validación de entrada: Idea Semilla vacía"; Estado Error ; alerta en Slack	Rama false de ? ¿Datos completos?, fila en Log de Errores vinculada
5	Camino infeliz: idea demasiado corta	Idea "Hola" (4 caracteres)	—	Error "Idea Semilla demasiado corta (4 < 10)"	Idem
6	Camino infeliz: categoría sin contexto	Categoría que no existe en el Manual (ej. "Legales")	—	Error "Sin contexto validado..."; no se gastan tokens	Rama false de ? ¿Hay contexto RAG?
7	Fallo de API de IA	Cambiar temporalmente <code>cfg.modeloIA</code> a <code>modelo-inexistente</code>	—	3 reintentos, después Log "🤖 IA · Gemini..." con el 404; Estado Error	Nodo con "Retry on fail" y salida de error
8	Lote (varios registros juntos)	3 filas en "Generando" a la vez	Aprobar 2 y rechazar 1	El loop procesa de a uno; cada pieza tiene su propia pausa HITL	Ejecución con 3 iteraciones
9	Anti-bucle	Editar una fila que ya está "En revisión"	—	El trigger no la vuelve a tomar	Lista de ejecuciones sin disparos nuevos

6.2 Evidencias (capturas) a incluir en el documento público

1. Canvas completo del workflow en n8n, con el flujo principal y los subworkflows a la vista.
2. Zoom al nodo **HITL** (configuración *Send and Wait*, aprobación doble y límite de espera).
3. Lista de *Executions* con al menos una ejecución en **Waiting** y 5 o más ejecuciones terminadas (éxito y error).
4. Mensaje de Slack con los botones Aprobar/Rechazar y la respuesta en el hilo.
5. Post publicado en #contenido-publicado.
6. Tabla Contenidos con registros en Publicado, Rechazado y Error.

- 7. Tabla Log de Errores con registros vinculados a Contenidos.
- 8. Dashboard (Shared View) abierto en incógnito, con los 3 KPIs.
- 9. Credenciales de n8n ocultas: se muestran solo los nombres, nunca las keys.

6.3 Resultados de las corridas (1 de octubre de 2026, n8n Cloud)

#	Hora	Caso	Resultado observado	Estado final en Airtable
1	11:21	Credencial de Airtable sin acceso a la base (403)	Error output del Anti-bucle → SUB 4 → alerta en Slack "Forbidden". Se corrigió con un Personal Access Token	—
2	11:30	Fallo real de la API de IA (Gemini 503 <i>Service unavailable</i>)	3 reintentos automáticos; luego SUB 4: alerta en Slack y Estado Error	Error
3	11:38	Camino feliz + HITL : borrador generado con RAG y enviado a revisión	La ejecución quedó en " Esperando " hasta el clic; al aprobar, SUB 2 publicó en #contenido-publicado	Publicado
4	11:48	Rechazo humano ("Cómo fijar tus tarifas como freelancer")	Clic en ✗ Rechazar → SUB 3, aviso en el hilo; nada en el canal de salida	Rechazado
5	11:52	Camino infeliz: Idea Semilla vacía	La IA no se invocó; registro en Log de Errores (vinculado al contenido) y alerta en Slack "Datos faltantes: Idea Semilla vacía"	Error

Las 5 corridas cubren el camino feliz, el rechazo, la pausa HITL, el dato faltante y el fallo de API. Las capturas de cada una están en el documento público.

Captura: workflow importado en n8n (flujo principal y subworkflows)

